

EasyFlow2 Analysis Workflow User Guide

A static English R Markdown manual and example based on the current Function01-07 modules and EasyFlow analysis script

James Lab

2026-05-09

Contents

1	Purpose Of This Document	2
2	Codebase Overview	3
3	Global Sections That Must Be Edited Before Running	3
3.1	Project Paths, Function Directory, And Data Directory	3
3.2	Required Packages	4
3.3	Automatic Loading Of Core Function Modules	5
3.4	Panel Mapping	5
3.5	API Key And Proxy Configuration	6
4	Recommended Centralized Configuration	7
5	Standard Workflow	9
5.1	Step 1: Data Reading, QC, And Preprocessing	9
5.2	Step 2: Interactive Gating	11
5.3	Step 3: Dimensionality Reduction And Clustering	12
5.4	Step 4: Dual-Track AI Annotation	14
5.5	Step 5: Central Control Panel For Plotting And Reporting Variables	17
5.6	Step 6: One-Click Publication-Ready Figure Generation	18
5.7	Embed Pre-Generated Figures In The Static PDF Manual	25
5.8	Optional Large-Scale Machine-Learning Projection	33
5.9	Step 7: Statistical Analysis And Report Output	39

6	Typical Output Files Generated By The Current Workflow	44
7	Suggested Methods Text For A Manuscript	44
8	Practical Notes And High-Frequency Pitfalls	45
8.1	<code>function_dir</code> and <code>setwd()</code> are environment-specific	45
8.2	<code>Plot_Scatterhist_Line()</code> requires additional packages	45
8.3	<code>Perform_Marker_DE()</code> currently supports two-group comparisons only	46
8.4	Panel consistency is essential before large-scale projection	46
8.5	<code>ADThorm-lite</code> requires a valid <code>Batch</code> column	46
8.6	HTML rendering requires Pandoc	46
9	Minimal Reproducible Example	46
10	Closing Note	48
11	Session Information	48

1 Purpose Of This Document

All code chunks in this document are provided as non-executing examples only. Automatic execution is globally disabled with `eval = FALSE`, so knitting to PDF produces a static manual instead of rerunning the workflow.

This document was rewritten directly from the current local EasyFlow codebase:

- 01_Preprocessing.R
- 02_Interactive_UI.R
- 03_DimReduction.R
- 04_Annotation.R
- 05_MachineLearning.R
- 06_Visualization.R
- 07_Stats_and_Export.R
- EasyFlow_Analysis.R

The goal of this document is to provide a fully English, publication-friendly R Markdown manual that can be used for:

1. day-to-day execution of the EasyFlow workflow;
2. lab onboarding and internal handoff;
3. project archiving and reproducibility;
4. manuscript methods writing and supplementary documentation.

This version follows the real structure of the current scripts rather than an external helper wrapper, and every adjustable location is annotated with parameter meanings and practical guidance.

2 Codebase Overview

File	Main role	Typical outputs
01_Preprocessing.R	FCS import, QC, compensation, transformation, downsampling, panel mapping, gating-template filtering	preprocessed cell-level data frame
02_Interactive_UI.R	interactive gating, gating-tree recording, threshold template export	<code>gated_cells</code> , gating tree CSV, threshold template CSV, gating tree PDF
03_DimReduction.R	clustering, t-SNE, UMAP, benchmark recording	<code>res_obj</code>
04_Annotation.R	rule-based annotation, negative-marker rule matching, LLM-assisted annotation, label normalization	<code>final_results</code> , <code>Cluster_Annotation_Report.csv</code>
05_MachineLearning.R	large-scale atlas projection and rapid new-sample projection	projected result object, RF model, selected features
06_Visualization.R	publication-ready visualization	ggplot objects, heatmap object, sample-level PCA
07_Stats_and_Export.R	abundance statistics, marker DE, trajectory inference, FCS export, HTML report generation	CSV, PNG, FCS, HTML
EasyFlow_Analysis.R	end-to-end master script	full analysis workflow template

3 Global Sections That Must Be Edited Before Running

3.1 Project Paths, Function Directory, And Data Directory

The current project-specific path block can be written as follows:

```
project_dir <- "/Volumes/1 TB/lly go colitis facs/EasyFlow/R_20260506"
function_dir <- project_dir
data_dir <- "/Volumes/1 TB/lly go colitis facs/20240619 colitis cytokine/mln"
figure_dir <- file.path(project_dir, "figures")
output_dir <- "/Volumes/Extreme_Pro/Codex/outputs"
gating_tree_pdf <- file.path(project_dir, "00_Gating_Tree_Plot.pdf")
trajectory_dir <- file.path(project_dir, "09_Trajectory_Outputs")

setwd(project_dir)
dir.create(figure_dir, recursive = TRUE, showWarnings = FALSE)
dir.create(output_dir, recursive = TRUE, showWarnings = FALSE)
```

3.1.1 Adjustable parameters in this block

Parameter	Meaning	When to edit
project_dir	root working directory for the project	always edit for a new project
function_dir	directory containing the 01-07 function scripts	edit if the functions are stored in a subfolder
data_dir	directory containing the FCS files	always edit for a new dataset
figure_dir	output directory for PDF/PNG figures	edit if you want a different figure output path
output_dir	output directory for RDS/CSV/HTML results	edit if you want a different result output path

3.2 Required Packages

Recommended package installation and loading:

```
pkgs <- c(
  "shiny", "plotly", "miniUI", "dplyr", "Rtsne", "ggplot2",
  "stringr", "ggpubr", "scales", "RColorBrewer", "igraph", "RANN",
  "ggribes", "pheatmap", "httr2", "jsonlite", "hexbin", "viridis",
  "flowCore", "uwot", "ranger", "BiocManager", "flowStats",
  "reshape2", "tidyr", "ggalluvial", "ggrepel", "patchwork", "cowplot"
)

new_pkgs <- pkgs[!(pkgs %in% installed.packages()[, "Package"])]
if (length(new_pkgs)) install.packages(new_pkgs)

if (!requireNamespace("FlowSOM", quietly = TRUE)) {
  BiocManager::install("FlowSOM")
}
if (!requireNamespace("slingshot", quietly = TRUE)) {
  BiocManager::install(c("slingshot", "SingleCellExperiment"))
}

invisible(lapply(c(pkgs, "FlowSOM"), library, character.only = TRUE))
```

3.2.1 Adjustable parameters in this block

Parameter	Meaning	When to edit
pkgs	list of CRAN dependencies	edit if you add or remove visualization/statistics features

Parameter	Meaning	When to edit
FlowSOM	optional dependency for FlowSOM clustering	required when <code>cluster_method = "flowsom"</code>
slingshot	optional dependency for trajectory inference	required when running <code>Infer_Trajectory()</code>
patchwork / cowplot	required by the marginal-density scatter plot	required when using <code>Plot_Scatterhist_Line()</code>

3.3 Automatic Loading Of Core Function Modules

```
r_files <- list.files(function_dir, pattern = "[0-9]{2}_.*\\.Rr$", full.names = TRUE)

message(">>> Loading EasyFlow core modules...")
for (file in sort(r_files)) {
  source(file)
  message("  Loaded: ", basename(file))
}
message(">>> EasyFlow core modules loaded successfully.")
```

3.3.1 Adjustable parameters in this block

Parameter	Meaning	When to edit
pattern	determines which R files are loaded	edit if your naming convention changes
sort(r_files)	loads modules in prefix order	edit if you want manual control of load order

3.4 Panel Mapping

```
my_panel <- c(
  "[live/dead fv-ef780]-A" = "L/D",
  "BV605-A" = "CD4",
  "BV510-A" = "CD103",
  "FITC-A" = "GM-CSF",
  "PE-Cy7-A" = "IL17a",
  "BV421-A" = "TNFa",
  "PE-A" = "IL6",
  "AF700-A" = "IL10",
  "APC-A" = "IFNg",
  "eFluor450-A" = "Foxp3"
)
```

3.4.1 Adjustable parameters in this block

Parameter	Meaning	When to edit
my_panel	mapping from raw cytometer channel names to analysis marker names	must be edited for every experimental panel
left-hand names	actual channel names in the FCS files	must match the acquisition panel exactly
right-hand names	marker names used downstream in analysis	edit when standardizing naming style

3.4.2 Panel mapping recommendations

1. Use a consistent naming style across the project, for example all uppercase human-style names or all mixed-case marker names.
2. Annotation rules, plotting markers, and statistical markers must exactly match the mapped column names.
3. If downstream steps reference IFNg, TNFa, or Foxp3, the mapped column names must match those strings exactly.

3.5 API Key And Proxy Configuration

```
Sys.setenv(OPENROUTER_API_KEY = "")
Sys.setenv(http_proxy = "http://127.0.0.1:7890")
Sys.setenv(https_proxy = "http://127.0.0.1:7890")

tryCatch({
  res <- jsonlite::fromJSON("https://ipapi.co/json/")
  cat("Current R session IP:", res$ip, "\n")
  cat("Current R session region:", res$city, ",", res$country_name, "\n")
}, error = function(e) cat("Network test failed. Proxy or internet access may be unavailable.\n"))
```

3.5.1 Adjustable parameters in this block

Parameter	Meaning	When to edit
OPENROUTER_API_KEY	API key used for LLM-assisted annotation	required when using the LLM path in <code>Annotate_Clusters_Dual_Track()</code>
http_proxy	HTTP proxy	only set this if your local environment requires a proxy
https_proxy	HTTPS proxy	only set this if your local environment requires a proxy

3.5.2 Publication note

For a manuscript-ready or shareable version of this document, keep the API key as a placeholder rather than embedding a real secret in the file.

4 Recommended Centralized Configuration

Although the original master script spreads parameters across multiple steps, long-term maintenance is much easier if the parameters are defined centrally first and then reused downstream. The following block maps directly onto the current `EasyFlow Analysis.R` logic.

```
group_keywords <- c("wt", "ko")

preprocess_cfg <- list(
  n_cells_per_sample = 50000,
  do_QC = TRUE,
  do_compensation = TRUE,
  transform_method = "arcsinh",
  cofactor = 150
)

cluster_cfg <- list(
  k_neighbors = 30,
  cluster_mode = "B",
  do_scale = TRUE,
  cluster_method = "louvain",
  flowsom_k = 20,
  tsne_perplexity = NULL,
  umap_n_neighbors = 15,
  umap_min_dist = 0.2
)

annotation_cfg <- list(
  model = "openai/gpt-5.4",
  use_llm_only = TRUE,
  ontology = c("CD4 T cell", "Unknown"),
  max_clusters_per_batch = 20,
  base_population_context = paste(
    "All cells in this analysis were pre-gated as CD4+ T cells",
    "from mLN.",
    "All annotations must stay within the CD4 T-cell universe."
  ),
  allow_functional_prefix = TRUE,
  unknown_threshold = 35,
  annotation_granularity = "fine"
)
```

```

projection_cfg <- list(
  batch1_folder = "/Volumes/1 TB/lly go colitis facs/20240619 colitis cytokine/mln",
  batch2_folder = "/Volumes/1 TB/lly go colitis facs/situmulate mlns",
  batch1_label = "Batch1_20240619_mLN",
  batch2_label = "Batch2_stimulated_mLN",
  gating_template_path = "/Volumes/Extreme_Pro/Codex/Gating_Thresholds_Template.csv",
  n_cells_per_sample = 1500000,
  target_node = "cell_live_cell_CD4",
  margin = 0.02,
  target_label = "Cluster_Name",
  cv_folds = 5,
  rejection_threshold = 0.55,
  use_class_weights = TRUE,
  rare_cell_boost = 2.0,
  batch_correction = "adtnorm_lite",
  batch_col = "Batch",
  reference_batch = "Batch1_20240619_mLN",
  batch_diagnostic_pdf = "ADTnorm_Batch_Correction_Diagnostics.pdf"
)

plot_order_cfg <- list(
  Group = c("wt", "ko")
)

plot_cfg <- list(
  my_dim = "UMAP",
  my_color = "Cluster_Name",
  my_split = "Group",
  my_target_marker = "IL17a",
  my_group_var = "Group",
  my_fill_var = "Group",
  my_facet_var = "Cluster_Name",
  my_bar_x = "Group",
  my_bar_fill = "Cluster_Name",
  my_sankey_axis1 = "Group",
  my_sankey_axis2 = "Cluster_Name",
  my_x_marker = "IL17a",
  my_y_marker = "Foxp3",
  my_density_grp = "Cluster_Name",
  my_exp_grp = "Cluster_Name",
  my_dot_markers = NULL,
  my_conf_true = "CellType",
  my_conf_pred = "Cluster_Name",
  my_sample_group = "Group",
  my_sample_id = "SampleID"
)

```



```
stats_cfg <- list(
  my_stat_target = "Cluster_Name",
  my_stat_group = "Group",
  my_stat_method = "wilcoxon",
  my_stat_trans = "clr"
)

report_cfg <- list(
  project_name = "EasyFlow2 Final Report",
  out_file = "EasyFlow2_Final_Report.html",
  template = "manuscript"
)
```

4.0.1 Meaning of each configuration block

Configuration block	Role
group_keywords	defines how experimental groups are inferred from filenames
preprocess_cfg	controls preprocessing, QC, compensation, transformation, and downsampling
cluster_cfg	controls clustering, t-SNE, and UMAP
annotation_cfg	controls rule-based annotation, LLM-assisted annotation, and label granularity
projection_cfg	controls two-batch large-scale projection, ADThnorm-lite correction, and rejection thresholding
plot_order_cfg	fixes plotting order of groups and cell populations
plot_cfg	controls the variables used in the figure generation section
stats_cfg	controls abundance statistics and transformation method
report_cfg	controls HTML report title and template

5 Standard Workflow

5.1 Step 1: Data Reading, QC, And Preprocessing

```
raw_data_rds <- file.path(output_dir, "raw_data.rds")

if (file.exists(raw_data_rds)) {
  raw_data <- readRDS(raw_data_rds)
  message("Loaded raw_data from: ", raw_data_rds)
}
```

```

} else {
  raw_data <- Prepare_Raw_Data(
    work_dir = data_dir,
    group_keywords = group_keywords,
    n_cells_per_sample = preprocess_cfg$n_cells_per_sample,
    do_QC = preprocess_cfg$do_QC,
    do_compensation = preprocess_cfg$do_compensation,
    transform_method = preprocess_cfg$transform_method,
    cofactor = preprocess_cfg$cofactor
  )
  raw_data <- Apply_Panel_Mapping(raw_data, my_panel)
  if (!dir.exists(output_dir)) dir.create(output_dir, recursive = TRUE)
  saveRDS(raw_data, file = raw_data_rds)
  message("Saved raw_data to: ", raw_data_rds)
}

head(raw_data)
cat(
  "Preprocessing completed:",
  nrow(raw_data), "cells across",
  length(unique(raw_data$SampleID)), "samples.\n"
)

```

5.1.1 Adjustable parameters in Prepare_Raw_Data()

Parameter	Current example value	Meaning	Practical guidance
work_dir	data_dir	directory containing the FCS files	must be changed to your real dataset path
group_keywords	c("wt", "ko")	labels groups using filename keywords	replace with the group keywords used in your study
n_cells_per_sample	50000	maximum number of cells retained per sample	20,000 to 50,000 is common for reference atlas construction
do_QC	TRUE	enables automated debris and doublet filtering	recommended for routine flow cytometry analyses
do_compensation	TRUE	applies the embedded compensation matrix from the FCS file	usually recommended for fluorescence flow cytometry
transform_method	"arcsinh"	transformation method, one of "arcsinh", "logicle", or "none"	arcsinh is usually the most robust default

Parameter	Current example value	Meaning	Practical guidance
cofactor	150	ArcSinh cofactor	typically 150 for flow cytometry and 5 for CyTOF

5.1.2 Adjustable parameters in Apply_Panel_Mapping()

Parameter	Meaning
df	the preprocessed data frame returned from the previous step
panel_list	the channel-to-marker mapping table

5.1.3 Typical outputs from this step

- raw_data
- SampleID
- Group
- mapped marker columns based on my_panel

5.2 Step 2: Interactive Gating

```
gated_cells_rds <- file.path(output_dir, "gated_cells.rds")
file.exists(file.path(output_dir, "gated_cells.rds"))
if (file.exists(gated_cells_rds)) {
  gated_cells <- readRDS(gated_cells_rds)
  message("Loaded gated_cells from: ", gated_cells_rds)
} else {
  gated_cells <- raw_data
  message("No gated_cells.rds found; using raw_data as fallback.")
}
head(gated_cells)
cat(
  "Gating completed:",
  nrow(gated_cells), "cells retained across",
  length(unique(gated_cells$CellType)), "annotated populations.\n"
)
```

```
if (file.exists(gating_tree_pdf)) {
  knitr::include_graphics(gating_tree_pdf)
} else {
  cat("Gating tree PDF was not found at:", gating_tree_pdf, "\n")
}
```

5.2.1 Parameters for Run_Universal_Gating()

Parameter	Meaning
merged_df	the preprocessed cell-level data frame

5.2.2 Adjustable parameters in Plot_Gating_Tree()

Parameter	Default	Meaning
csv_path	"Gating_Tree_Structure.csv"	path to the gating-tree structure CSV
save_path	"00_Gating_Tree_Plot.pdf"	output path for the gating-tree PDF

5.2.3 Typical output files from the gating step

File	Meaning
Gating_Thresholds_Template.csv	1% to 99% threshold template for each gated node across channels
Gating_Tree_Structure.csv	gating parent-child structure with cumulative counts
00_Gating_Tree_Plot.pdf	visualized gating tree

5.2.4 Adjustable elements during interactive gating

1. `x_axis` and `y_axis` determine which two markers are displayed for manual selection.
2. `Show density` controls whether density contours are overlaid.
3. `Set as global gate` keeps only the currently selected cells.
4. `Remove selection` removes the selected debris or unwanted events.
5. `Annotate selection` assigns a name to the selected subpopulation.

5.3 Step 3: Dimensionality Reduction And Clustering

```
ref_results_rds <- file.path(output_dir, "ref_results.rds")

if (file.exists(ref_results_rds)) {
  ref_results <- readRDS(ref_results_rds)
  message("Loaded ref_results from: ", ref_results_rds)
} else {
  ref_results <- Run_Analysis_On_Gated_Data(
    gated_data = gated_cells,
```

```

k_neighbors = cluster_cfg$k_neighbors,
cluster_mode = cluster_cfg$cluster_mode,
do_scale = cluster_cfg$do_scale,
cluster_method = cluster_cfg$cluster_method,
flowsom_k = cluster_cfg$flowsom_k,
tsne_perplexity = cluster_cfg$tsne_perplexity,
umap_n_neighbors = cluster_cfg$umap_n_neighbors,
umap_min_dist = cluster_cfg$umap_min_dist
)
if (!dir.exists(output_dir)) dir.create(output_dir, recursive = TRUE)
saveRDS(ref_results, file = ref_results_rds)
message("Saved ref_results to: ", ref_results_rds)
}

cat(
  "Clustering completed:",
  nrow(ref_results$Data), "cells and",
  length(unique(ref_results$Data$Cluster)), "clusters.\n"
)

```

5.3.1 Adjustable parameters in Run_Analysis_On_Gated_Data()

Parameter	Current example value	Meaning	Practical guidance
gated_data	gated_cells	the gated cell-level data	fixed output from the previous step
k_neighbors	30	number of neighbors used for graph construction	increase for larger or more complex datasets if needed
cluster_mode	"B"	"A" clusters previously named cells only, "B" clusters all gated-in cells	"B" is usually preferred for global atlas construction
do_scale	TRUE	applies Z-score scaling before clustering	recommended when markers are on different scales
cluster_method	"louvain"	one of "louvain", "leiden", or "flowsom"	Louvain or Leiden are usually the safest general choices
flowsom_k	20	number of FlowSOM meta-clusters	only used when cluster_method = "flowsom"
tsne_perplexity	NULL	t-SNE perplexity	leaving this as NULL lets the function choose a safe value automatically

Parameter	Current example value	Meaning	Practical guidance
<code>umap_n_neighbors</code>	15	number of UMAP neighbors	increase if you want smoother manifolds
<code>umap_min_dist</code>	0.2	UMAP minimum distance	decrease if you want tighter embeddings

5.3.2 Returned object structure

`ref_results` usually contains:

- `ref_results$Data`: cell-level data, cluster labels, t-SNE, and UMAP coordinates
- `ref_results$Info$UMAP_Model`: reusable UMAP model for downstream projection
- `ref_results$Info$Benchmarks`: runtime benchmarks

5.4 Step 4: Dual-Track AI Annotation

The legacy rule format is still supported:

```
custom_rules <- list(
  "CD4 T cell" = c("CD4"),
  "Treg" = c("CD4", "Foxp3"),
  "Th1" = c("CD4", "IFNg"),
  "Th17" = c("CD4", "IL17a")
)
```

The current annotation module is better used with explicit positive and negative marker logic:

```
custom_rules <- list(
  "CD4 T cell" = list(
    positive = c("CD4"),
    negative = c("CD8")
  ),
  "Treg" = list(
    positive = c("CD4", "Foxp3"),
    negative = c("IFNg", "TNFa")
  ),
  "Th1" = list(
    positive = c("CD4", "IFNg"),
    negative = c("Foxp3")
  ),
  "Th17" = list(
    positive = c("CD4", "IL17a"),
    negative = c("Foxp3")
  )
)
```

5.4.1 Recommended annotation call

```
final_results_rds <- file.path(output_dir, "final_results.rds")

if (file.exists(final_results_rds)) {
  final_results <- readRDS(final_results_rds)
  message("Loaded final_results from: ", final_results_rds)
} else {
  my_ontology <- c(
    "CD4 T cell",
    "Treg",
    "Th1",
    "Th17",
    "Unknown"
  )

  final_results <- Annotate_Clusters_Dual_Track(
    res_obj = ref_results,
    rules = custom_rules,
    model = annotation_cfg$model,
    use_llm_only = annotation_cfg$use_llm_only,
    ontology = annotation_cfg$ontology,
    max_clusters_per_batch = annotation_cfg$max_clusters_per_batch,
    base_population_context = annotation_cfg$base_population_context,
    allow_functional_prefix = annotation_cfg$allow_functional_prefix,
    unknown_threshold = annotation_cfg$unknown_threshold,
    annotation_granularity = annotation_cfg$annotation_granularity
  )
  if (!dir.exists(output_dir)) dir.create(output_dir, recursive = TRUE)
  saveRDS(final_results, file = final_results_rds)
  message("Saved final_results to: ", final_results_rds)
}

cat(
  "Annotation completed. Available label columns:",
  paste(
    intersect(
      c("Cluster_Name", "Coarse_Name", "Fine_Name", "Major_Lineage", "Minor_Lineage", "State_N",
        colnames(final_results$Data)
      ),
    collapse = ", "
  ),
  "\n"
)
```

5.4.2 Adjustable parameters in Annotate_Clusters_Dual_Track()

Parameter	Current example value	Meaning	Practical guidance
res_obj	ref_results	clustering result object	fixed output from the previous step
rules	custom_rules	rule-based annotation table	strongly recommended when canonical marker combinations are known
model	"openai/gpt-5.4"	LLM used for annotation	adjust according to your available API endpoint
use_llm_only	TRUE	skips rule-based matching and runs the full LLM path	set to FALSE if you want rules first, LLM second
ontology	c("CD4 T cell", "Unknown")	constrains allowed annotation labels	strongly recommended for publication-grade annotation
max_clusters_per_batch	20	number of clusters submitted per LLM batch	reduce if prompts become too large
base_population_context	CD4 T-cell context sentence	provides parent-lineage context to the LLM	highly recommended if upstream gating already defined a biological universe
allow_functional_prefix	TRUE	allows functional-state prefixes in labels	set to FALSE for more conservative naming
unknown_threshold	35	labels low-confidence clusters as Unknown	lower it to reduce Unknown labels; increase it for stricter annotation
annotation_granularity	fine	one of "fine", "coarse", or "both"	fine or coarse are most common in manuscripts

5.4.3 Manual review after annotation

```

# Run this block manually after reviewing Cluster_Annotation_Report.csv
corrected_report <- read.csv("Cluster_Annotation_Report.csv")
final_results$Data$Cluster_Name <- corrected_report$fine_name[
  match(final_results$Data$Cluster, corrected_report$cluster)
]
# Re-save after manual correction
saveRDS(final_results, file = file.path(output_dir, "final_results.rds"))

```


5.4.4 Typical outputs from this step

- Cluster_Annotation_Report.csv
- final_results\$Data\$Cluster_Name
- final_results\$Data\$Coarse_Name
- final_results\$Data\$Fine_Name
- final_results\$Data\$Major_Lineage
- final_results\$Data\$Minor_Lineage
- final_results\$Data\$State_Name

5.5 Step 5: Central Control Panel For Plotting And Reporting Variables

This section mirrors the central variable assignment area in the current master script.

```
my_dim <- plot_cfg$my_dim
my_color <- plot_cfg$my_color
my_split <- plot_cfg$my_split

my_target_marker <- plot_cfg$my_target_marker
my_group_var <- plot_cfg$my_group_var
my_fill_var <- plot_cfg$my_fill_var
my_facet_var <- plot_cfg$my_facet_var

my_bar_x <- plot_cfg$my_bar_x
my_bar_fill <- plot_cfg$my_bar_fill
my_sankey_axis1 <- plot_cfg$my_sankey_axis1
my_sankey_axis2 <- plot_cfg$my_sankey_axis2

my_x_marker <- plot_cfg$my_x_marker
my_y_marker <- plot_cfg$my_y_marker
my_density_grp <- plot_cfg$my_density_grp

my_exp_grp <- plot_cfg$my_exp_grp
my_dot_markers <- plot_cfg$my_dot_markers

my_conf_true <- plot_cfg$my_conf_true
my_conf_pred <- plot_cfg$my_conf_pred

my_sample_group <- plot_cfg$my_sample_group
my_sample_id <- plot_cfg$my_sample_id

final_results <- Set_Plot_Order(
  final_results,
  order_list = plot_order_cfg
)
```

5.5.1 Adjustable parameters in this section

Parameter	Meaning
<code>my_dim</code>	chooses "UMAP" or "tSNE" as the main embedding view
<code>my_color</code>	determines how the main embedding is colored, for example <code>Cluster</code> or <code>Cluster_Name</code>
<code>my_split</code>	optional faceting variable, for example <code>Group</code> ; set to <code>NULL</code> for no splitting
<code>my_target_marker</code>	marker used in violin, bar, and ridge plots
<code>my_group_var</code>	grouping variable for expression comparisons
<code>my_fill_var</code>	fill variable for violin-style plots
<code>my_facet_var</code>	faceting variable for violin/bar plots
<code>my_bar_x</code>	x-axis variable for composition plots
<code>my_bar_fill</code>	fill variable for composition plots
<code>my_sankey_axis1</code> / <code>my_sankey_axis2</code>	axes used in the alluvial plot
<code>my_x_marker</code> / <code>my_y_marker</code>	two markers used in the dual-marker scatter plot
<code>my_density_grp</code>	grouping variable used in the dual-marker scatter plot
<code>my_exp_grp</code>	grouping variable for dotplot and heatmap
<code>my_dot_markers</code>	optional custom marker list for the dotplot; <code>NULL</code> enables automatic detection
<code>my_conf_true</code> / <code>my_conf_pred</code>	true-label and predicted-label columns for the confusion matrix
<code>my_sample_group</code> / <code>my_sample_id</code>	color and label variables for sample-level PCA
<code>plot_order_cfg</code>	explicit global plotting order, strongly recommended for publication figures

5.6 Step 6: One-Click Publication-Ready Figure Generation

5.6.1 Main embedding figure

```
p_dim <- Plot_Dim_Reduction(  
  final_results,  
  dim_method = my_dim,  
  color_by = my_color,  
  split_by = my_split  
)  
  
print(p_dim)  
  
ggsave(  
  paste0("01_", my_dim, "_by_", my_color,  
    ifelse(is.null(my_split), "", paste0("_split_", my_split)),
```

```

        ".pdf"),
    plot = p_dim,
    width = 12,
    height = 6
)

```

5.6.1.1 Adjustable parameters in Plot_Dim_Reduction()

Parameter	Default	Meaning
dim_method	"UMAP" / "tSNE"	chooses which 2D embedding to display
color_by	"Cluster"	variable used for coloring
split_by	NULL	optional faceting variable

5.6.2 Composition plots

```

p_comp_fill <- Plot_Composition(
  final_results,
  x_var = my_bar_x,
  fill_var = my_bar_fill,
  position = "fill",
  correct_sample_bias = TRUE
)

p_comp_stack <- Plot_Composition(
  final_results,
  x_var = my_bar_x,
  fill_var = my_bar_fill,
  position = "stack",
  correct_sample_bias = TRUE
)
print(p_comp_fill)

```

5.6.2.1 Adjustable parameters in Plot_Composition()

Parameter	Default	Meaning
x_var	"Group"	x-axis grouping variable
fill_var	"Cluster_Name"	stacked fill variable
position	"fill" or "stack"	proportional or absolute stacked plot
correct_sample_bias	TRUE	computes proportions per sample before averaging by group

Parameter	Default	Meaning
sample_col	"SampleID"	sample identifier column

5.6.3 Alluvial plot

```
p_sankey <- Plot_Alluvial(
  final_results,
  axis1 = my_sankey_axis1,
  axis2 = my_sankey_axis2,
  fill_var = my_bar_fill
)

print(p_sankey)
```

5.6.3.1 Adjustable parameters in Plot_Alluvial()

Parameter	Default	Meaning
axis1	"Group"	first axis
axis2	"Cluster_Name"	second axis
fill_var	"Cluster_Name"	variable controlling flow colors

5.6.4 Dotplot

```
p_dot <- Plot_Dotplot(
  final_results,
  markers = my_dot_markers,
  group_by = my_exp_grp
)

print(p_dot)
```

5.6.4.1 Adjustable parameters in Plot_Dotplot()

Parameter	Default	Meaning
markers	NULL	custom marker list; NULL auto-detects numeric marker columns
group_by	"Cluster_Name"	grouping variable shown on the y-axis

5.6.5 All-in-one violin plot

The current master script has already switched from a manually written violin block to `Plot_Violin_AllInOne()`. This is one of the main upgrades that should be highlighted in the documentation.

```
p_v <- Plot_Violin_AllInOne(  
  res_obj = final_results,  
  marker = my_target_marker,  
  cluster_by = my_facet_var,  
  group_by = my_group_var,  
  fill_by = my_fill_var,  
  add_boxplot = TRUE,  
  trim = FALSE,  
  add_stats = TRUE,  
  stat_method = "wilcox.test",  
  stat_label = "p.signif",  
  hide_ns = FALSE,  
  p_y_offset = 0.08,  
  logfc_y_offset = 0.16,  
  logfc_digits = 2  
)  
  
print(p_v)
```

5.6.5.1 Adjustable parameters in `Plot_Violin_AllInOne()`

Parameter	Current example value	Meaning
marker	controlled by <code>my_target_marker</code>	marker whose expression distribution will be shown
cluster_by	"Cluster_Name"	first-level grouping variable
group_by	"Group"	second-level grouping variable inside each cluster
fill_by	NULL, usually set to <code>group_by</code>	violin fill variable
add_boxplot	TRUE	overlays boxplots on violins
trim	FALSE	trims violin tails or not
add_stats	TRUE	adds statistical comparison labels
stat_method	"wilcox.test"	one of "wilcox.test" or "t.test"
stat_label	"p.signif"	significance stars or formatted p-values; use "p.format" if needed
comparisons	NULL	usually left at default in the current workflow

Parameter	Current example value	Meaning
hide_ns	FALSE	hides non-significant labels if set to TRUE
violin_alpha	0.7	violin transparency
logfc_digits	2	number of decimal places shown in log2FC
logfc_prefix	"log2FC="	prefix used for the logFC label
p_y_offset	0.08	relative height of p-value labels
logfc_y_offset	0.16	relative height of logFC labels
point_size	NULL	optionally overlays jitter points

5.6.6 LogFC-annotated barplot

```
p_bar <- Compare_Populations_Barplot(
  final_results,
  marker = my_target_marker,
  group_by = my_group_var,
  reference_group = "wt",
  show_errorBars = TRUE,
  logfc_threshold = 0.1,
  p_threshold = 0.05,
  facet_by = my_facet_var
)

p_bar
```

5.6.6.1 Adjustable parameters in Compare_Populations_Barplot()

Parameter	Current example value	Meaning
marker	my_target_marker	marker to compare
group_by	my_group_var	grouping variable used for comparison
reference_group	"wt"	reference group for logFC calculation
show_errorBars	TRUE	displays SEM/error bars
logfc_threshold	0.1	only labels logFC above this threshold
p_threshold	0.05	only labels statistically significant comparisons
facet_by	my_facet_var	optional faceting variable

5.6.7 Heatmap

```
hm_res <- Plot_Heatmap(  
  final_results,  
  group_by = my_exp_grp,  
  scale = "row",  
  cluster_rows = TRUE,  
  cluster_cols = TRUE  
)  
  
if (!is.null(hm_res$Plot$gtable)) {  
  grid::grid.newpage()  
  grid::grid.draw(hm_res$Plot$gtable)  
}
```

5.6.7.1 Adjustable parameters in Plot_Heatmap()

Parameter	Default	Meaning
group_by	"Cluster"	grouping variable used before aggregation
scale	"row"	row scaling, column scaling, or none
cluster_rows	TRUE	clusters rows if TRUE
cluster_cols	TRUE	clusters columns if TRUE
color_palette	NULL	custom heatmap palette
show_border	TRUE	shows tile borders
fontsize	10	text size

5.6.8 Confusion matrix

```
if (my_conf_true %in% colnames(final_results$Data) &&  
    my_conf_pred %in% colnames(final_results$Data)) {  
  p_conf <- Plot_Confusion_Matrix(  
    final_results,  
    true_label = my_conf_true,  
    pred_label = my_conf_pred  
  )  
  
  p_conf  
} else {  
  cat("Confusion matrix skipped because the requested columns were not found.\n")  
}
```

5.6.8.1 Adjustable parameters in Plot_Confusion_Matrix()

Parameter	Default	Meaning
true_label	"CellType"	ground-truth label column
pred_label	"Cluster_Name"	predicted label column

5.6.9 Ridge plot

```
p_density <- Plot_Density_Line(
  final_results,
  marker = my_target_marker,
  group_by = my_group_var
)

p_density
```

5.6.9.1 Adjustable parameters in Plot_Density_Line()

Parameter	Default	Meaning
marker	required	marker used for density estimation
group_by	"Cluster_Name"	grouping variable
custom_colors	NULL	custom color palette

5.6.10 Dual-marker scatter plot with marginal densities

```
p_scatterhist <- Plot_Scatterhist_Line(
  final_results,
  x_marker = my_x_marker,
  y_marker = my_y_marker,
  group_by = my_density_grp
)

p_scatterhist
```

5.6.10.1 Adjustable parameters in Plot_Scatterhist_Line()

Parameter	Current default	Meaning
x_marker	required	x-axis marker
y_marker	required	y-axis marker
group_by	"Cluster_Name"	grouping variable used for colors
custom_colors	NULL	custom color vector

Parameter	Current default	Meaning
point_size	0.5	main scatter-point size
point_alpha	0.6	main scatter-point transparency
density_linewidth	0.6	marginal density line width
margin_ratio	0.15	relative size of marginal panels
density_height	0.8	controls how tall the marginal density curves appear

5.6.11 Sample-level PCA

```
p_sample_pca <- Plot_Sample_PCA(
  final_results,
  color_by = my_sample_group,
  label_by = my_sample_id
)

p_sample_pca
```

5.6.11.1 Adjustable parameters in Plot_Sample_PCA()

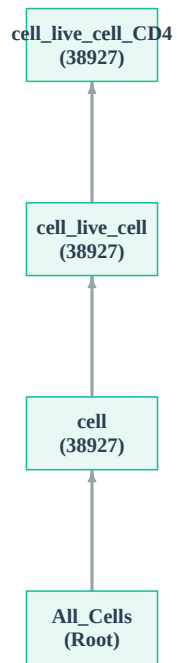
Parameter	Default	Meaning
color_by	"Group"	grouping variable used for sample coloring
label_by	"SampleID"	sample label column

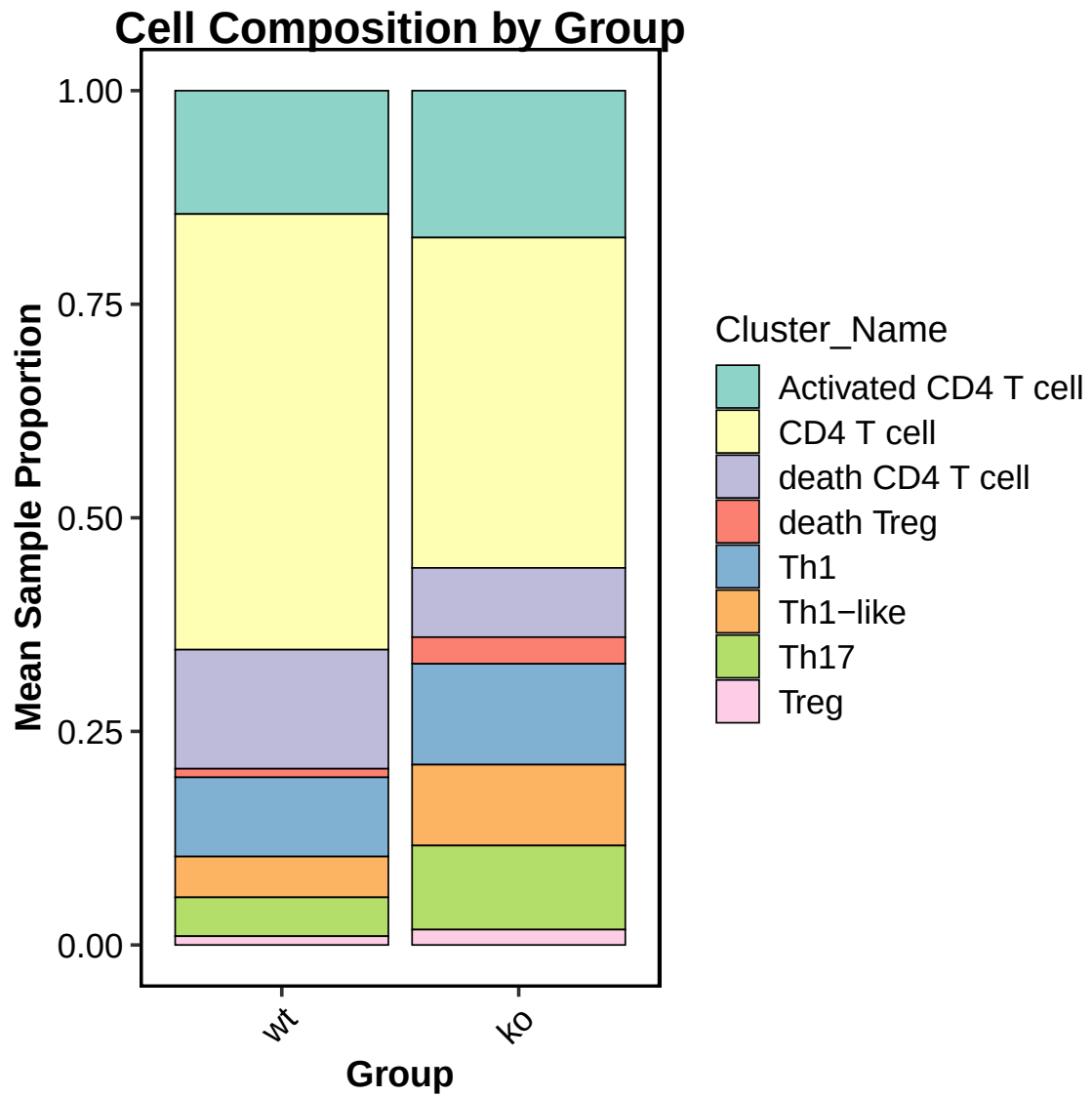
5.7 Embed Pre-Generated Figures In The Static PDF Manual

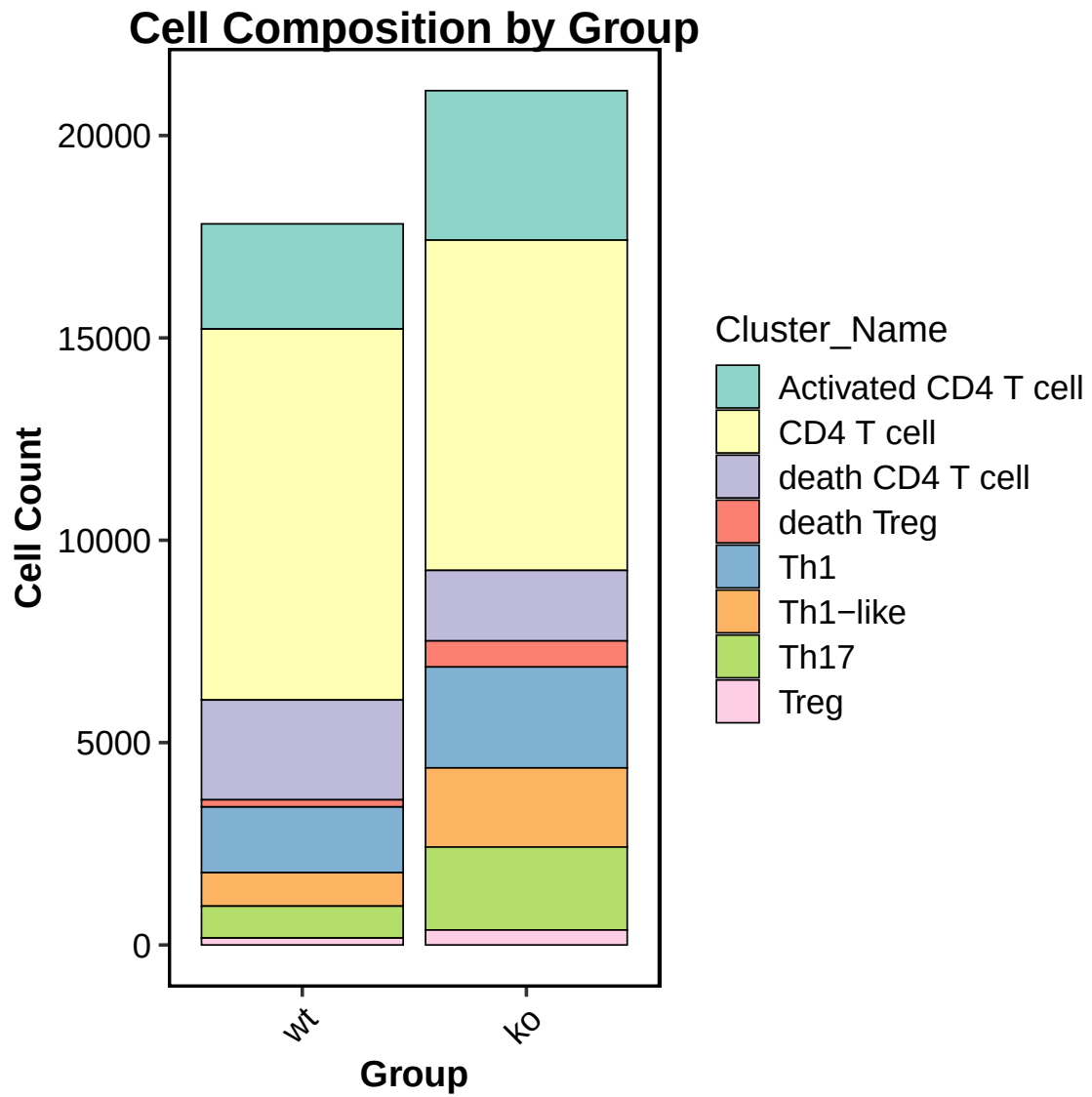
Because this document is configured as a static manual with global `eval = FALSE`, the plotting chunks above are displayed as reproducible example code but are not executed during PDF export. Therefore, those chunks will not automatically create or embed figures.

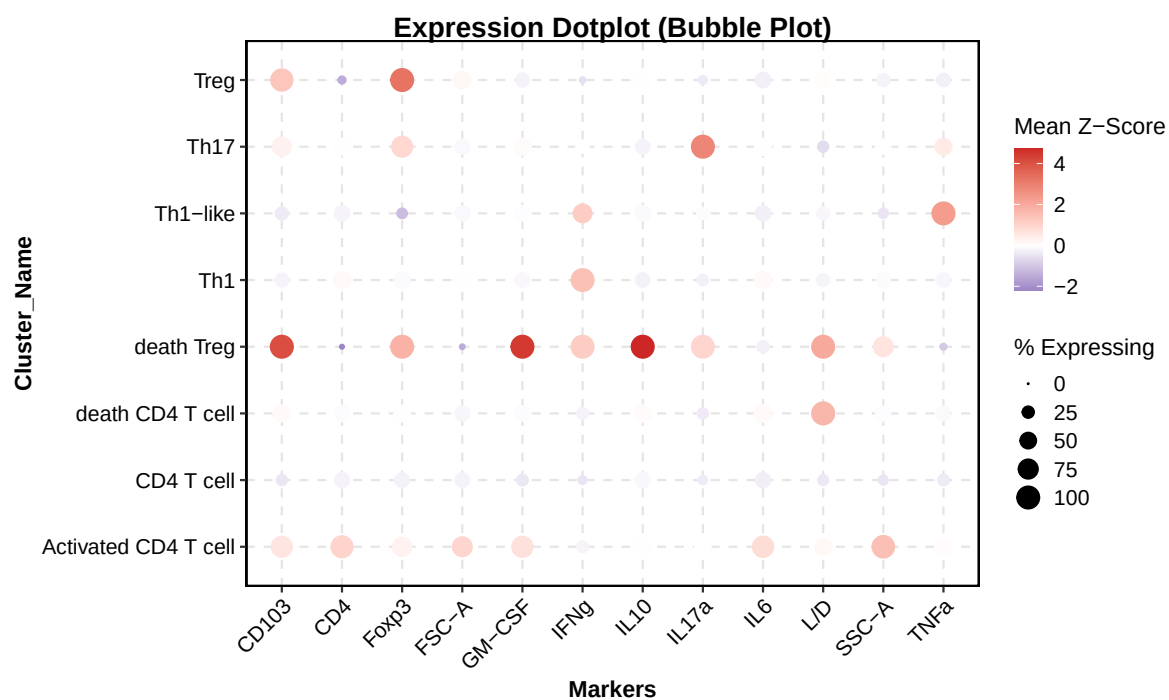
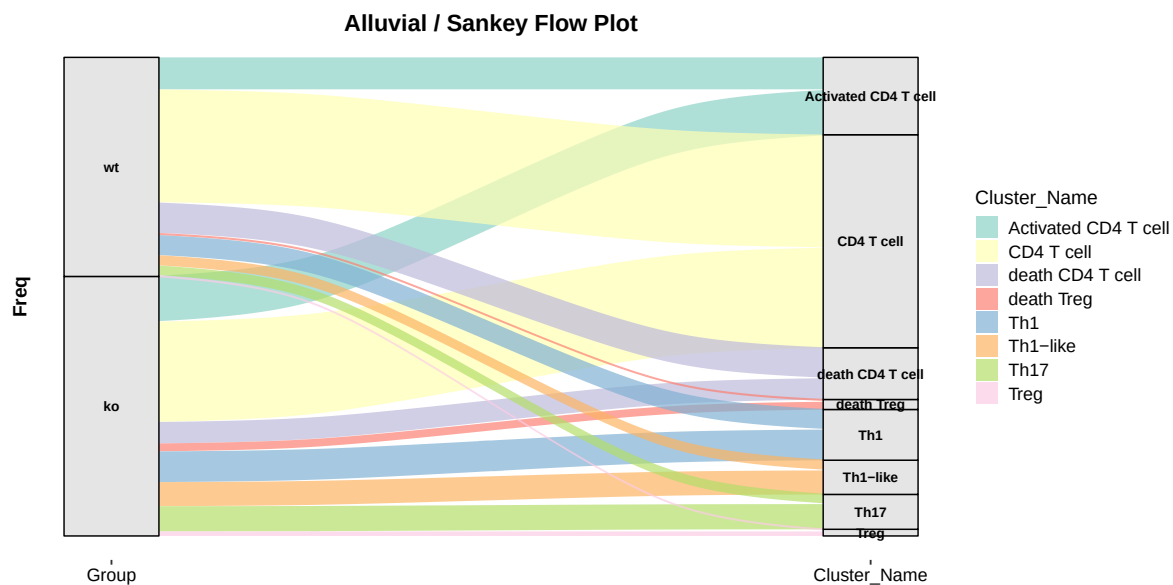
To include figures without rerunning the analysis, embed the already generated PDF or PNG files explicitly with `knitr::include_graphics()`. This chunk is intentionally set to `eval = TRUE` because it only reads existing image files; it does not rerun preprocessing, clustering, annotation, projection, or statistics.

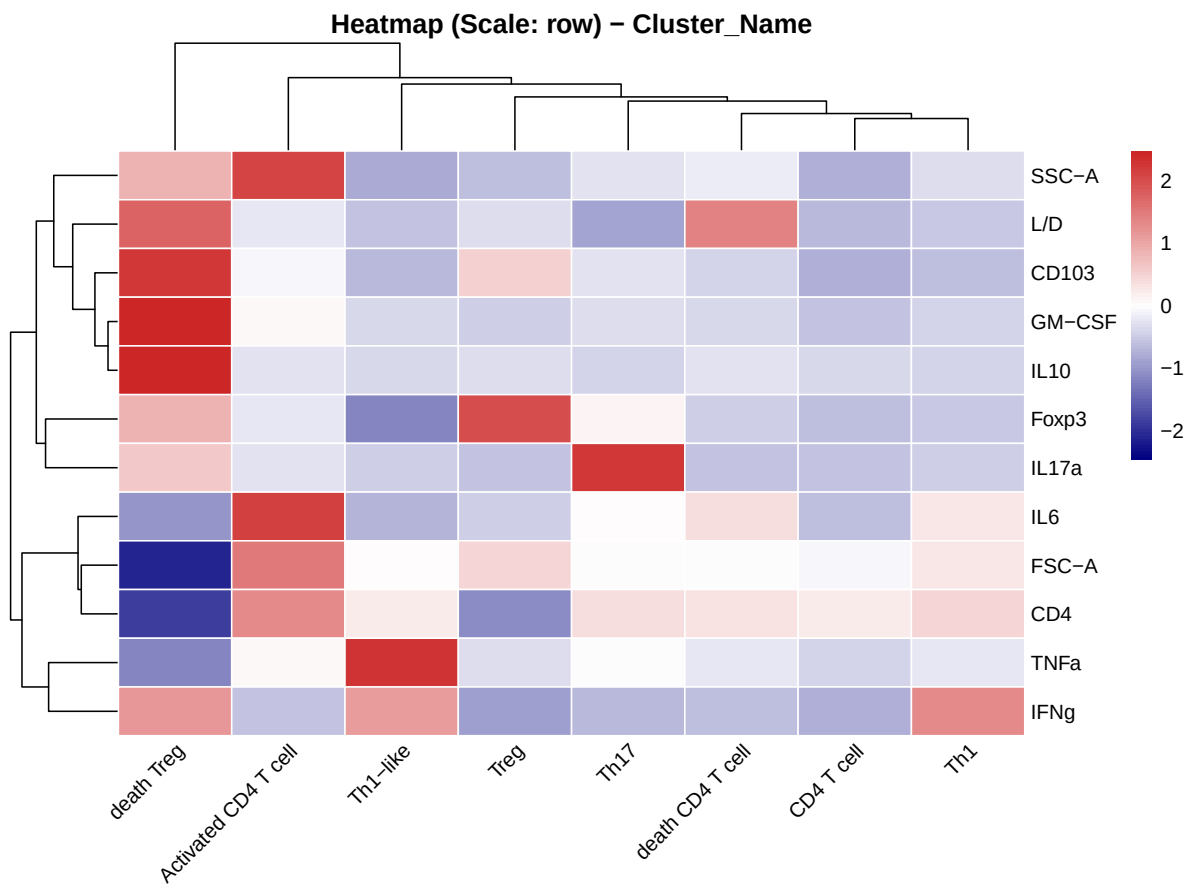
Cell Gating Hierarchy Tree

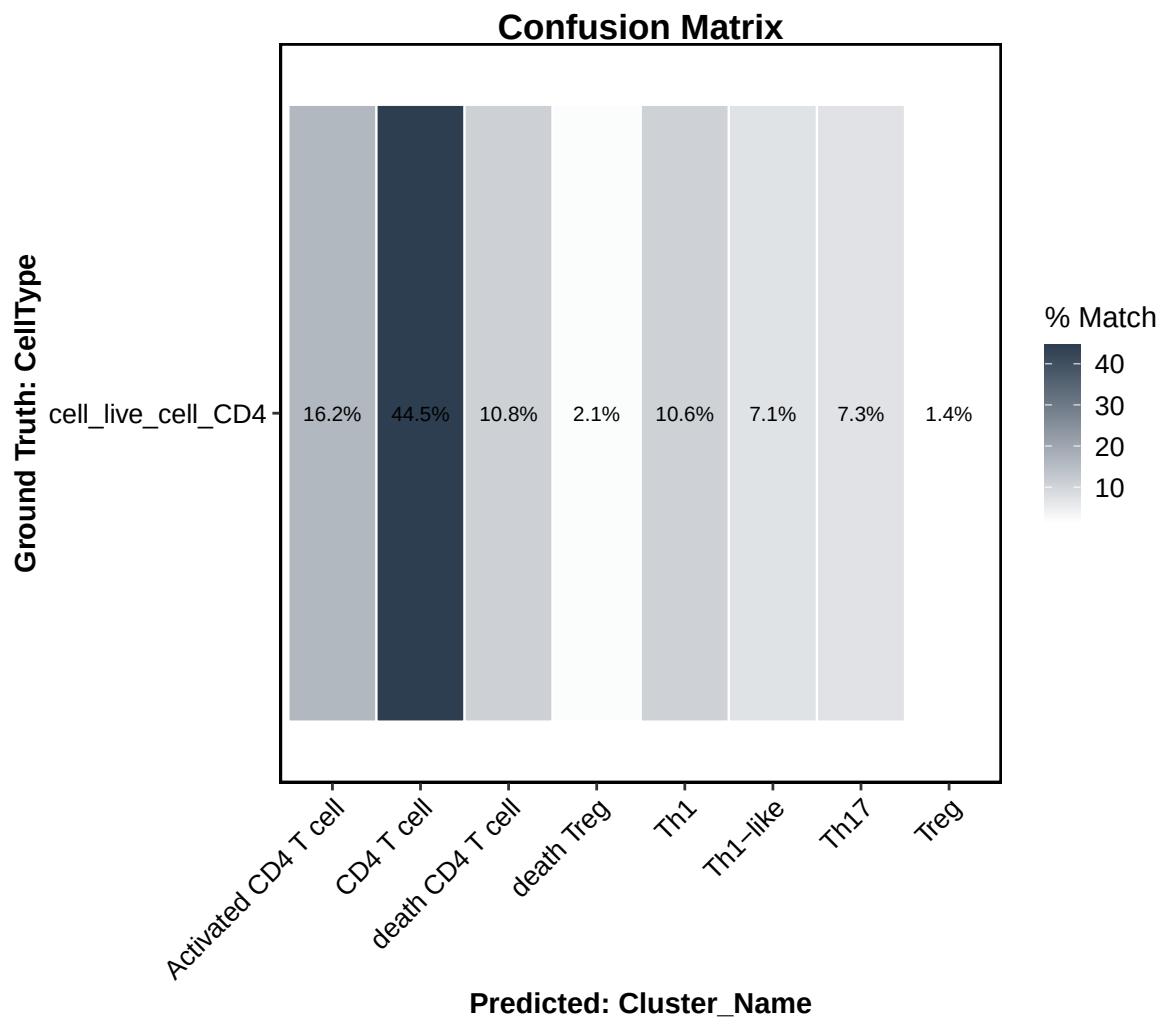


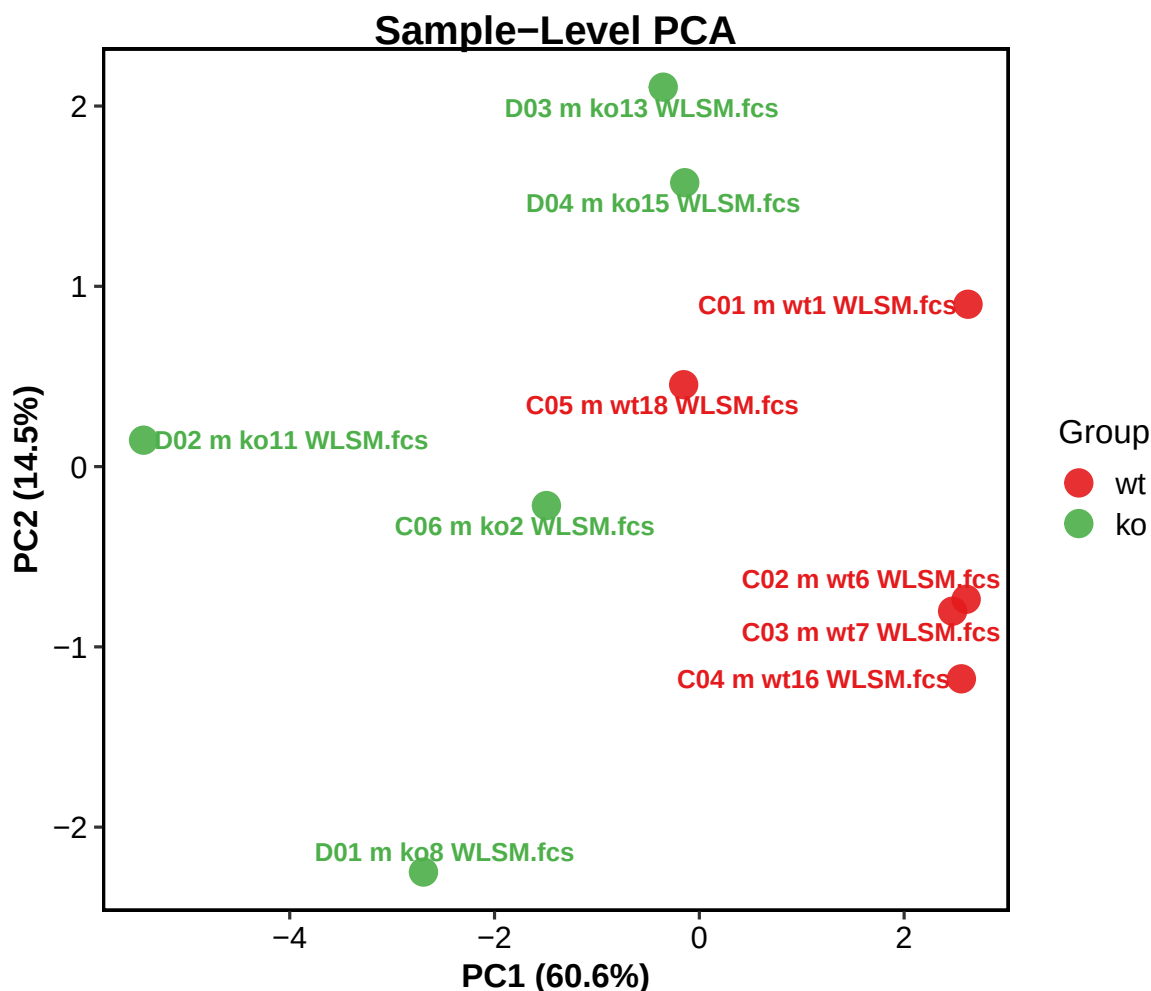












5.7.1 Adjustable parameters in the static figure-embedding chunk

Parameter	Meaning	Practical guidance
<code>static_figure_dir</code>	folder containing already generated figures	edit this to the real output folder before exporting the PDF
<code>static_figure_files</code>	ordered list of figure filenames to embed	remove figures you do not want, or add newly generated figures
<code>eval = TRUE</code>	allows only this lightweight embedding chunk to run	safe because it only reads image files and does not rerun the analysis
<code>out.width</code>	displayed width of each embedded figure	use "95%" for full-page figures, or smaller values for compact manuals

5.8 Optional Large-Scale Machine-Learning Projection

This section has intentionally been placed after the publication-ready figure section, as requested. It remains an optional extension of the core workflow and is most useful when a high-quality reference atlas has already been finalized and you want to project labels onto very large datasets.

5.8.1 Why ADThnorm-lite is included here

The current `05_MachineLearning.R` includes an ADThnorm-inspired correction option named `adtnorm_lite`. It is designed for large-scale projection across multiple flow/CITE-like protein batches. Before the random-forest projection step, ADThnorm-lite aligns marker-wise expression distributions across batches by matching density-derived landmarks. If density peaks and valleys cannot be matched reliably, the function falls back to robust quantile landmark alignment.

In practical terms, this option is useful when the same panel was acquired in more than one experimental batch and direct projection produces visible batch separation. The correction is applied only to the numeric marker features used by the model; metadata columns such as `Batch`, `Group`, `SampleID`, `Cluster_Name`, and UMAP coordinates are ignored.

Important requirement: the updated `05_MachineLearning.R` must be sourced before running this section. If R reports `unused arguments (batch_correction = ..., batch_col = ...)`, the session is still using an older `05_MachineLearning.R`.

5.8.2 Prepare two projection batches

The following block mirrors the current `EasyFlow Analysis.R` section titled `# Step 4b (optional): large-scale machine-learning projection`. The two folders represent two acquisition batches. Each batch is preprocessed and panel-mapped separately, then a `Batch` column is added before merging.

```
batch1_folder <- projection_cfg$batch1_folder
batch2_folder <- projection_cfg$batch2_folder

batch1_label <- projection_cfg$batch1_label
batch2_label <- projection_cfg$batch2_label
gating_template_path <- projection_cfg$gating_template_path

batch1_raw_data <- Prepare_Raw_Data(
  work_dir = batch1_folder,
  group_keywords = group_keywords,
  n_cells_per_sample = projection_cfg$n_cells_per_sample,
  do_QC = preprocess_cfg$do_QC,
  do_compensation = preprocess_cfg$do_compensation,
  transform_method = preprocess_cfg$transform_method,
  cofactor = preprocess_cfg$cofactor
)
batch1_raw_data <- Apply_Panel_Mapping(batch1_raw_data, my_panel)
batch1_raw_data$Batch <- batch1_label
```

```

batch2_raw_data <- Prepare_Raw_Data(
  work_dir = batch2_folder,
  group_keywords = group_keywords,
  n_cells_per_sample = projection_cfg$n_cells_per_sample,
  do_QC = preprocess_cfg$do_QC,
  do_compensation = preprocess_cfg$do_compensation,
  transform_method = preprocess_cfg$transform_method,
  cofactor = preprocess_cfg$cofactor
)
batch2_raw_data <- Apply_Panel_Mapping(batch2_raw_data, my_panel)
batch2_raw_data$Batch <- batch2_label

```

5.8.3 Adjustable parameters for two-batch preparation

Parameter	Current example value	Meaning	Practical guidance
batch1_folder	"/Volumes/1 TB/1ly go colitis facs/20240619 colitis cytokine/mln"	folder containing the first batch of FCS files	edit for the reference or baseline acquisition batch
batch2_folder	"/Volumes/1 TB/1ly go colitis facs/situmulate mlns"	folder containing the second batch of FCS files	edit for the additional acquisition batch
batch1_label	"Batch1_20240619_mLN"	label written to Batch for batch 1	use a short, stable, publication-friendly label
batch2_label	"Batch2_stimulated_mLN"	label written to Batch for batch 2	use the same naming style as <code>batch1_label</code>
group_keywords	c("wt", "ko")	filename keywords used to infer biological groups	edit when the group names differ
n_cells_per_sample	1500000	maximum number of cells sampled per FCS file for projection	reduce for testing; increase for final large-scale projection
Batch	created manually	metadata column identifying acquisition batch	required when <code>batch_correction =</code> <code>"adtnorm_lite"</code>

5.8.4 Recommended template-gating call

```

batch1_raw_data <- Apply_Gating_Template(
  df = batch1_raw_data,

```

```

target_node = projection_cfg$target_node,
template_path = gating_template_path,
margin = projection_cfg$margin
)

batch2_raw_data <- Apply_Gating_Template(
  df = batch2_raw_data,
  target_node = projection_cfg$target_node,
  template_path = gating_template_path,
  margin = projection_cfg$margin
)

massive_raw_data <- dplyr::bind_rows(batch1_raw_data, batch2_raw_data)
massive_raw_data$Batch <- factor(
  massive_raw_data$Batch,
  levels = c(batch1_label, batch2_label)
)

```

5.8.5 Adjustable parameters in Apply_Gating_Template()

Parameter	Current example value	Meaning	Practical guidance
df	batch1_raw_data or batch2_raw_data	large dataset to be filtered	apply the same template independently to each batch before merging
target_node	"cell_live_cell_CD4"	gating-template node used as the global filter	must exactly match the node name exported from the gating template
template_path	projection_cfg\$gating_template_path	gating template	use an absolute path when knitting from another working directory
margin	0.02	shrinks the template boundaries by this amount	increase for stricter filtering, but too much may remove true cells

5.8.6 Main large-scale projection call with ADThorm-lite

```

final_massive_results <- Project_Atlas_To_Massive_Data(
  ref_obj = final_results,
  massive_data = massive_raw_data,
  target_label = projection_cfg$target_label,

```

```

cv_folds = projection_cfg$cv_folds,
rejection_threshold = projection_cfg$rejection_threshold,
use_class_weights = projection_cfg$use_class_weights,
rare_cell_boost = projection_cfg$rare_cell_boost,
batch_correction = projection_cfg$batch_correction,
batch_col = projection_cfg$batch_col,
reference_batch = batch1_label,
batch_diagnostic_pdf = NULL
)

cat(
  "Large-scale projection completed:",
  nrow(final_massive_results$Data), "cells projected.\n"
)

```

Expected ADTnorm-lite outputs:

- `ADTnorm_Lite_Batch_Correction_Diagnostics.csv`: marker-by-marker correction summary, written to the current working directory by default.
- `ADTnorm_Batch_Correction_Diagnostics.pdf`: before/after density plots for each corrected marker, written to the current working directory in the example above.
- `final_massive_results$Info$Batch_Correction`: stored diagnostic information when available.

5.8.7 Optional plotting of projected results

```

final_massive_results <- Set_Plot_Order(
  final_massive_results,
  order_list = list(
    Group = c("wt", "ko"),
    Batch = c(batch1_label, batch2_label)
  )
)

projection_dim <- "UMAP"
projection_color <- "Cluster_Name"
projection_split <- "Batch"

p_dim_projected <- Plot_Dim_Reduction(
  final_massive_results,
  dim_method = projection_dim,
  color_by = projection_color,
  split_by = projection_split
)

```

```

out_dim_name <- paste0(
  "01_", projection_dim, "_by_", projection_color,
  "_split_", projection_split,
  "_ADTnormLite.pdf"
)

p_dim_projected
ggsave(out_dim_name, plot = p_dim_projected, width = 12, height = 6)

```

5.8.8 Adjustable parameters in Project_Atlas_To_Massive_Data()

Parameter	Current example value	Meaning	Practical guidance
ref_obj	final_results	annotated reference atlas	should be a high-quality, well-annotated reference object
massive_data	massive_raw_data	large target dataset to predict	fixed output from the previous step
target_label	"Cluster_Name"	label column to learn and project	can be changed to CellType or another valid label column
cv_folds	5	number of CV folds	set to 1 if you want to skip cross-validation
rejection_threshold	0.55	probability threshold below which cells become Unassigned	increase for stricter rejection, decrease for more aggressive assignment
use_class_weights	TRUE	enables rare-class weighting	recommended when rare populations matter biologically
rare_cell_boost	2.0	strength of rare-class protection	larger values protect rare populations more strongly but may also increase misassignment risk
batch_correction	"adtnorm_lite"	enables marker-wise batch alignment before projection	set to "none" for single-batch data or when no batch correction is desired
batch_col	"Batch"	metadata column identifying the acquisition batch	must exist in massive_data before projection

Parameter	Current example value	Meaning	Practical guidance
reference_batch	"Batch1_20240619_mLN"	batch used as the expression-scale reference	usually choose the cleaner, baseline, or reference acquisition batch
batch_diagnostic_pdf	"ADTnorm_Batch_Correction_Diagnostics.pdf"	output PDF showing before/after density diagnostics	inspect this file to confirm that correction is biologically reasonable

5.8.9 How to interpret ADTnorm-lite diagnostics

The diagnostic PDF should show each marker's density distribution before and after correction. Good correction usually reduces broad batch shifts while preserving biologically meaningful positive and negative populations. If the after-correction densities look over-flattened or biologically implausible, rerun the projection with:

```
projection_cfg$batch_correction <- "none"
```

and compare UMAPs, dotplots, and sample-level abundance patterns between the corrected and uncorrected results.

5.8.10 Rapid projection for future new samples

```
if (exists("new_sample_data")) {
  new_sample_result <- Project_New_Sample(
    ref_obj = final_massive_results,
    new_data = new_sample_data,
    rejection_threshold = projection_cfg$rejection_threshold
  )
} else {
  message("new_sample_data was not found; skipping rapid new-sample projection.")
}
```

5.8.11 Adjustable parameters in Project_New_Sample()

Parameter	Current example value	Meaning
ref_obj	final_massive_results	projection object containing the trained RF model
new_data	new_sample_data	new sample-level data frame
target_label	"Cluster_Name"	output label column

Parameter	Current example value	Meaning
rejection_threshold	0.55	low-confidence rejection threshold

5.9 Step 7: Statistical Analysis And Report Output

5.9.1 Differential abundance statistics

```
stats_results_rds <- file.path(output_dir, "stats_results.rds")

if (file.exists(stats_results_rds)) {
  stats_results <- readRDS(stats_results_rds)
  message("Loaded stats_results from: ", stats_results_rds)
} else {
  stats_results <- Perform_Abundance_Stats(
    res_obj = final_results,
    target_label = stats_cfg$my_stat_target,
    group_col = stats_cfg$my_stat_group,
    method = stats_cfg$my_stat_method,
    transform = stats_cfg$my_stat_trans
  )
  if (!dir.exists(output_dir)) dir.create(output_dir, recursive = TRUE)
  saveRDS(stats_results, file = stats_results_rds)
  message("Saved stats_results to: ", stats_results_rds)
}

if (!is.null(stats_results$Stats) && nrow(stats_results$Stats) > 0) {
  knitr::kable(stats_results$Stats, caption = "Abundance statistics results")
} else if (is.data.frame(stats_results)) {
  knitr::kable(stats_results, caption = "Abundance statistics results")
} else {
  print(stats_results)
}

cat("Abundance testing completed:", nrow(stats_results$Stats), "rows in the statistics table.\n")
```

5.9.1.1 Adjustable parameters in Perform_Abundance_Stats()

Parameter	Current example value	Meaning	Practical guidance
res_obj	final_results	final result object	fixed output from the previous step
target_label	"Cluster_Name"	label used to define populations	Cluster_Name is the most common choice

Parameter	Current example value	Meaning	Practical guidance
group_col	"Group"	sample-level grouping variable	must be a valid sample-level metadata column
method	"wilcoxon"	one of "wilcoxon", "t.test", "kruskal", "anova", or "lmm"	Wilcoxon or t-test are common for two-group comparisons
transform	"clr"	one of "none", "asin_sqrt", or "clr"	clr is often preferred for compositional abundance data
random_effect	NULL	random-effect column for mixed models	only used when method = "lmm"
sample_col	"SampleID"	sample ID column	usually does not need to be changed

5.9.2 Optional marker differential expression analysis

```

de_results_rds <- file.path(output_dir, "de_results.rds")

if (file.exists(de_results_rds)) {
  de_results <- readRDS(de_results_rds)
  message("Loaded de_results from: ", de_results_rds)
} else {
  de_results <- tryCatch(
    Perform_Marker_DE(
      res_obj = final_results,
      group_col = stats_cfg$my_stat_group,
      cluster_col = "Cluster_Name",
      method = "wilcoxon"
    ),
    error = function(e) { message("Marker DE failed: ", e$message); NULL }
  )
  if (!is.null(de_results)) {
    saveRDS(de_results, file = de_results_rds)
    message("Saved de_results to: ", de_results_rds)
  }
}

if (!is.null(de_results) && nrow(de_results) > 0) {
  knitr::kable(head(de_results, 30), caption = "Marker differential expression results (top 30)")
  cat("Marker differential analysis completed:", nrow(de_results), "rows exported.\n")
}

```

5.9.2.1 Adjustable parameters in Perform_Marker_DE()

Parameter	Default	Meaning
cluster_name	NULL	analyze one cluster only, or all clusters when NULL
group_col	"Group"	grouping variable used for the comparison
cluster_col	"Cluster_Name"	cluster label column
markers	NULL	custom marker list; NULL triggers automatic detection
method	"wilcoxon" / "t.test"	two-group marker comparison method

5.9.3 Trajectory inference

```

traj_res_rds <- file.path(output_dir, "traj_res.rds")

if (file.exists(traj_res_rds)) {
  traj_res <- readRDS(traj_res_rds)
  final_results <- traj_res$res_obj
  message("Loaded traj_res from: ", traj_res_rds)
} else {
  traj_res <- tryCatch(
    Infer_Trajectory(
      res_obj = final_results,
      start_cluster = NULL,
      use_dim = "UMAP",
      out_dir = trajectory_dir
    ),
    error = function(e) { message("Trajectory inference failed: ", e$message); NULL }
  )
  if (!is.null(traj_res)) {
    final_results <- traj_res$res_obj
    saveRDS(traj_res, file = traj_res_rds)
    message("Saved traj_res to: ", traj_res_rds)
  }
}

if (!is.null(traj_res)) {
  cat(
    "Trajectory inference completed. Output files:",
    paste(unlist(traj_res$plot_files), collapse = ", "),
    "\n"
  )
}

```

```

if (exists("traj_res") && !is.null(traj_res)) {
  trajectory_files <- unlist(traj_res$plot_files)
  trajectory_files <- trajectory_files[file.exists(trajectory_files)]
  if (length(trajectory_files) > 0) {
    knitr::include_graphics(trajectory_files)
  } else {
    cat("Trajectory figures were not found in:", trajectory_dir, "\n")
  }
}

```

5.9.3.1 Adjustable parameters in Infer_Trajectory()

Parameter	Default	Meaning
start_cluster	NULL	explicitly defines the starting cluster
start_by_col	NULL	infers the starting state from an annotation column such as CellType
start_by_value	NULL	the annotation value used together with start_by_col
start_mode	"majority" or "centroid"	how the starting state is mapped to a cluster when needed
use_dim	"UMAP"	embedding used for trajectory inference
out_dir	"09_Trajectory_Outputs"	output directory for trajectory figures
prefix	"Trajectory"	filename prefix
point_size	0.5	point size in the exported trajectory plots
seed	123	random seed

5.9.4 FCS export

```

Export_To_FCS(final_results, out_dir = "09_Final_FCS_Output")

```

5.9.4.1 Adjustable parameters in Export_To_FCS()

Parameter	Default	Meaning
out_dir	"Annotated_FCS_Outputs"	directory for exported annotated FCS files
sample_col	"SampleID"	sample ID column

5.9.5 HTML report generation

```
Generate_Final_HTML_Report(  
  res_obj = final_results,  
  stats_res = stats_results,  
  project_name = report_cfg$project_name,  
  out_file = report_cfg$out_file,  
  template = report_cfg$template  
)  
  
Generate_Final_HTML_Report(  
  res_obj = final_results,  
  stats_res = stats_results,  
  out_file = "EasyFlow2_Review_Report.html",  
  template = "review"  
)  
  
Generate_Final_HTML_Report(  
  res_obj = final_results,  
  stats_res = stats_results,  
  out_file = "EasyFlow2_Custom_Report.html",  
  include_plots = c(  
    "sample_counts", "embedding", "composition",  
    "marker_dotplot", "trajectory", "abundance"  
  )  
)  
  
saveRDS(final_results, file.path(output_dir, "final_results.rds"))  
cat("Final result object saved to:", file.path(output_dir, "final_results.rds"), "\n")
```

5.9.5.1 Adjustable parameters in Generate_Final_HTML_Report()

Parameter	Default	Meaning
res_obj	final_results	final result object
stats_res	stats_results	abundance statistics result
project_name	"EasyFlow2 Final Report"	report title
out_file	"EasyFlow2_Final_Report.html"	HTML output filename
sample_col	"SampleID"	sample column name
group_col	"Group"	group column name
template	"manuscript"	one of "manuscript", "review", or "full"
include_plots	NULL	custom selection of plots to include
top_markers_n	10	number of highly variable markers shown in the report

Parameter	Default	Meaning
top_stats_n	15	number of top statistical results shown in the report
true_label	"CellType"	true-label column used in the confusion matrix
pred_label	"Cluster_Name"	predicted-label column used in the confusion matrix

5.9.5.2 Allowed include_plots values

Value	Meaning
sample_counts	retained cell counts per sample
embedding	UMAP/t-SNE overview
composition	group-level composition
marker_dotplot	marker identity dotplot
sample_pca	sample-level PCA
trajectory	trajectory and pseudotime section
abundance	abundance statistics section
confusion	confusion matrix section
metadata	pipeline metadata section

6 Typical Output Files Generated By The Current Workflow

Stage	File or directory	Content
gating	Gating_Thresholds_Template.csv	gating threshold template
gating	Gating_Tree_Structure.csv	gating tree structure
gating	00_Gating_Tree_Plot.pdf	gating tree PDF
annotation	Cluster_Annotation_Report.csv	cluster annotation report
figures	01_*.pdf to 10_*.pdf	publication-ready figures
statistics	08_Differential_Abundance_Statistics.csv	differential abundance results
marker DE	DE_Markers_*.csv	marker differential-expression results
trajectory	09_Trajectory_Outputs/	pseudotime output figures
FCS	09_Final_FCS_Output/	annotated FCS exports
report	EasyFlow2_Final_Report.html and related files	automated HTML reports

7 Suggested Methods Text For A Manuscript

The following paragraph can be adapted to your specific biological system:

Raw FCS files were processed using a custom R-based workflow, EasyFlow2, which integrates preprocessing, interactive hierarchical gating, unsupervised clustering, annotation, machine-learning projection, and downstream reporting. During preprocessing, FCS files were imported without instrument-level transformation, followed by optional quality control, fluorescence compensation, nonlinear transformation, and per-sample downsampling. Quality control included low-end FSC-A filtering to remove debris and FSC-A/FSC-H-based singlet selection. Compensation matrices were automatically extracted from the SPILL or spillover fields of the FCS header, and fluorescence channels were transformed using either `arcsinh` or `logicle` transformation. Preprocessed single-cell matrices were then subjected to custom interactive hierarchical gating, in which users iteratively defined cell populations in arbitrary two-dimensional marker space. Parent-child relationships among gated populations were explicitly recorded as a gating tree, and percentile-based thresholds for each node were exported as reusable gating templates. Downstream unsupervised analysis included graph-based or FlowSOM clustering together with t-SNE and UMAP embedding. Cluster identities were further assigned using a dual-track annotation framework that combined rule-based marker logic with LLM-assisted semantic annotation under ontology and lineage-context constraints. For large-scale cross-batch projection, an annotated reference atlas was used to train a random-forest classifier with optional rare-class weighting and out-of-distribution rejection. When multiple acquisition batches were projected together, marker-wise batch effects were optionally reduced using an ADThorm-inspired landmark-alignment procedure (`adtnorm_lite`) before label projection and UMAP coordinate transfer. The final outputs were used for population abundance testing, marker-level differential analysis, trajectory inference, annotated FCS export, and automated HTML report generation.

8 Practical Notes And High-Frequency Pitfalls

8.1 `function_dir` and `setwd()` are environment-specific

The absolute paths in the current project script are machine-specific. In a new project, the first items to update are usually:

- `setwd()`
- `function_dir`
- `my_folder` or `data_dir`

8.2 `Plot_Scatterhist_Line()` requires additional packages

If the dual-marker marginal-density scatter plot fails, first check:

- `patchwork`
- `cowplot`

These packages may not have been included in older versions of the setup block.

8.3 Perform_Marker_DE() currently supports two-group comparisons only

If `group_col` contains more than two groups, the current implementation will stop. For multi-group studies, either split the comparison into pairwise analyses or use another statistical framework.

8.4 Panel consistency is essential before large-scale projection

`Project_Atlas_To_Massive_Data()` and `Project_New_Sample()` both enforce strict feature matching between the reference atlas and new data. If a panel mismatch error appears, check:

1. whether `Apply_Panel_Mapping()` has been applied;
2. whether marker naming and capitalization are perfectly consistent;
3. whether the reference atlas and new samples truly come from the same panel.

8.5 ADTnorm-lite requires a valid Batch column

When `batch_correction = "adtnorm_lite"`, the merged projection dataset must contain a batch column such as `Batch`, and `reference_batch` must exactly match one of its values. If R reports unused arguments for `batch_correction`, `batch_col`, or `reference_batch`, restart the R session and source the updated `05_MachineLearning.R`.

8.6 HTML rendering requires Pandoc

If `Generate_Final_HTML_Report()` or the full R Markdown rendering fails, verify that Pandoc is available in your RStudio or system environment.

9 Minimal Reproducible Example

If you only want a compact working template, start from the following minimal example:

```
project_dir <- "/path/to/project"
data_dir <- "/path/to/fcs_folder"
setwd(project_dir)

for (file in sort(list.files(project_dir, pattern = "[0-9]{2}_.*\\.R$", full.names = TRUE))) {
  source(file)
}

my_panel <- c(
  "BV605-A" = "CD4",
  "BV510-A" = "CD103",
  "FITC-A" = "GM-CSF",
  "PE-Cy7-A" = "IL17a",
  "BV421-A" = "TNFa",
```

```

"PE-A" = "IL6",
"AF700-A" = "IL10",
"APC-A" = "IFNg",
"eFluor450-A" = "Foxp3"
)

raw_data <- Prepare_Raw_Data(
  work_dir = data_dir,
  group_keywords = c("wt", "ko"),
  n_cells_per_sample = 50000,
  do_QC = TRUE,
  do_compensation = TRUE,
  transform_method = "arcsinh",
  cofactor = 150
)

raw_data <- Apply_Panel_Mapping(raw_data, my_panel)
gated_cells <- Run_Universal_Gating(raw_data)

ref_results <- Run_Analysis_On_Gated_Data(
  gated_data = gated_cells,
  k_neighbors = 30,
  cluster_mode = "B",
  do_scale = TRUE,
  cluster_method = "louvain",
  umap_n_neighbors = 15,
  umap_min_dist = 0.2
)

final_results <- Annotate_Clusters_Dual_Track(
  res_obj = ref_results,
  model = "openai/gpt-5.4",
  use_llm_only = TRUE,
  ontology = c("CD4 T cell", "Unknown"),
  base_population_context = "All cells were pre-gated as CD4+ T cells.",
  annotation_granularity = "fine"
)

stats_results <- Perform_Abundance_Stats(
  res_obj = final_results,
  target_label = "Cluster_Name",
  group_col = "Group",
  method = "wilcoxon",
  transform = "clr"
)

Generate_Final_HTML_Report(

```

```
res_obj = final_results,  
stats_res = stats_results,  
out_file = "EasyFlow2_Final_Report.html",  
template = "manuscript"  
)
```

10 Closing Note

The current EasyFlow codebase already supports a complete workflow from raw FCS input through interactive gating, dimensionality reduction, dual-track annotation, machine-learning projection, publication-grade visualization, statistical analysis, trajectory inference, and automated HTML reporting. In practice, the three most important aspects to maintain consistently are:

1. path and directory configuration;
2. panel naming and marker naming consistency;
3. consistent use of label columns across annotation, plotting, and statistics.

If these three components remain aligned, downstream tuning and reproducibility become much easier.

11 Session Information

```
sessionInfo()
```